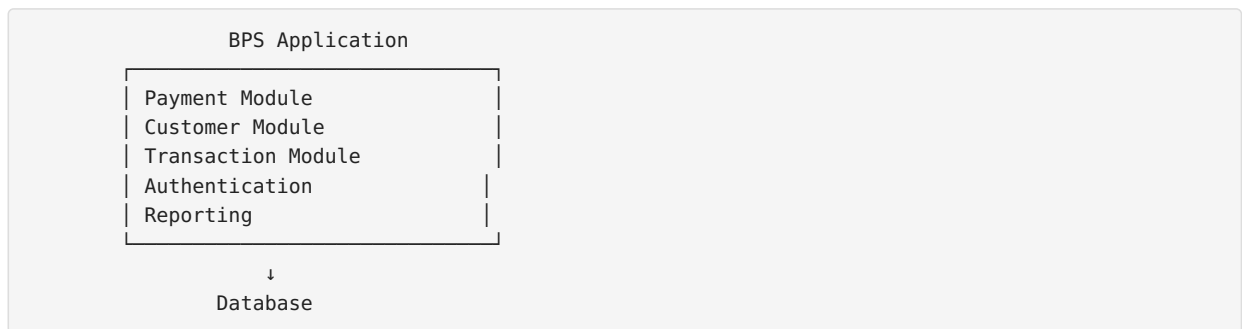


Microservices vs Monolithic Architecture

1. Monolithic Architecture

In a monolithic application, major business capabilities are deployed as **one application/unit**.

For example:



Everything may be packaged and deployed together:

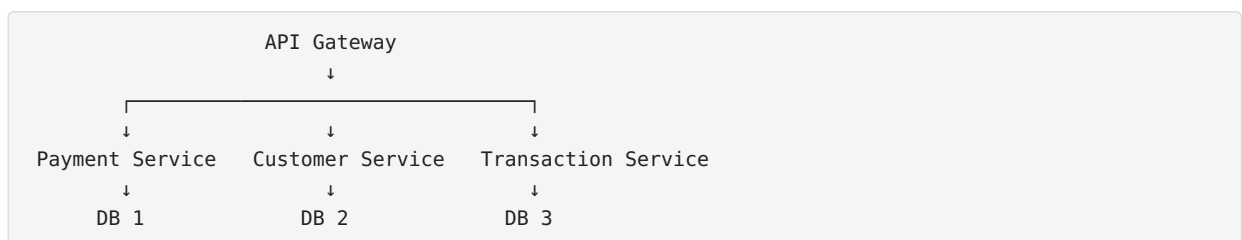
```
bps.jar
```

A change in one module often results in redeploying the whole application.

2. Microservices Architecture

The application is split into **independently deployable services**, typically around business capabilities.

Example:



Each service owns a specific responsibility and can often be developed, deployed, and scaled independently.

3. Main Difference

Monolith	Microservices
One deployable application	Multiple independently deployable services
Usually simpler initially	More operationally complex

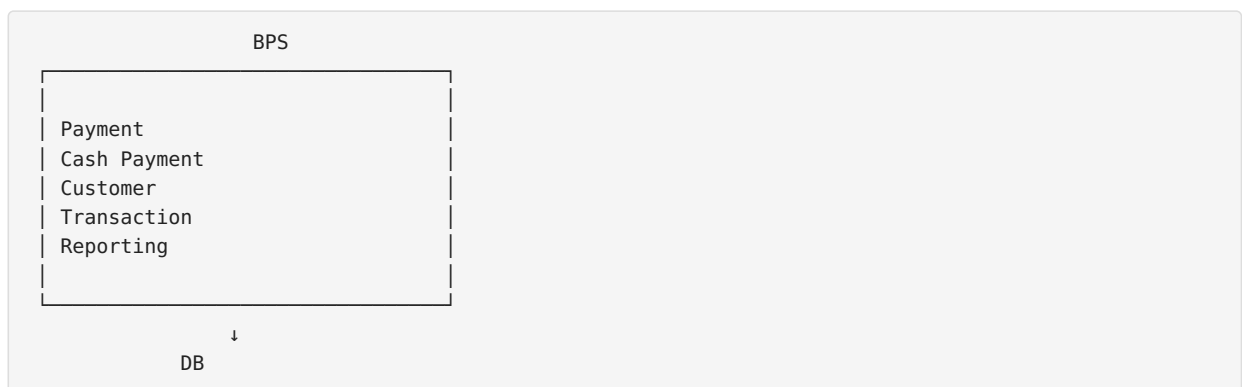
Often centralized database	Often service-owned data
Simple in-process calls	Network communication
Easier local debugging	Distributed debugging
Scale whole application	Scale individual services
Failure often contained within process	Network/service failures become important
Simpler transactions	Distributed consistency is harder

4. Monolith Example

Suppose BPS contains:

```
Payment
Cash Payment
Customer
Transaction
Reporting
```

Monolith:



Calling another module might simply be:

```
paymentService.process();
```

That's an **in-process method call**.

5. Microservices Example

Could be split into:

```
BPS
|
├─ Payment Service
├─ Transaction Service
├─ Customer Service
└─ Notification Service
```

Communication:

```
Payment Service
```

```
↓  
REST / Kafka  
↓  
Transaction Service
```

Now the call is a **network call**, not a local Java method call.

That creates new failure scenarios:

```
timeout  
connection failure  
HTTP 500  
service unavailable  
partial success  
retry  
duplicate request
```

6. Why Companies Use Microservices

Independent deployment

Suppose only the payment service changes.

With microservices:

```
Payment Service  
↓  
deploy only payment service
```

You don't necessarily need to redeploy unrelated services.

Independent scaling

Suppose payment processing receives much more traffic:

```
Payment Service → 10 instances  
Customer Service → 2 instances
```

You can scale according to workload.

Team ownership

Different teams can own different business capabilities.

Technology flexibility

In some organizations, different services can use different technologies where justified.

7. Why Not Always Use Microservices?

This is a **very important interview answer**.

Microservices aren't automatically better.

They introduce:

- network latency
- distributed failures
- service discovery/configuration
- observability complexity
- deployment complexity
- data consistency challenges
- distributed tracing
- more infrastructure
- more testing complexity

A small application may actually be easier and cheaper as a monolith.

Good interview answer

“I wouldn’t choose microservices simply because they’re modern. I’d choose them when independent scaling, deployment, team ownership, or domain boundaries justify the additional operational and distributed-system complexity.”

8. Database Difference

Monolith

Often:

```
All modules
  ↓
One database
```

Example:

```
Payment → DB
Customer → DB
Transaction → DB
```

Microservices

A common microservice principle is:

A service owns its data.

Example:

```
Payment Service
  ↓
Payment DB

Customer Service
  ↓
Customer DB
```

A service shouldn’t casually query another service’s database directly.

Instead:

```
Payment Service
  ↓
Customer Service API
```

or an event/message depending on the design.

9. Why Separate Databases?

Because otherwise you get:

```
Service A
Service B
Service C
  ↓
Same DB
```

and all services become tightly coupled to the same schema.

Then changing a table can break multiple services.

Service-owned data provides stronger boundaries.

But it also makes things harder:

A transaction spanning multiple service databases is no longer a simple local database transaction.

10. Distributed Transactions

This is very important for your **payment project**.

Monolith:

```
@Transactional
  ↓
DB update A
DB update B
DB update C
```

One database transaction can often handle all of them.

Microservices:

```
Payment Service
  ↓
Payment DB

Transaction Service
  ↓
Transaction DB
```

You cannot simply rely on one local `@Transactional` annotation to atomically commit both independent databases.

Now you may need:

- Saga pattern
- events
- idempotency
- retry
- compensation
- reconciliation
- outbox pattern

This is a high-value 3-year interview area.

11. Synchronous Communication

Example:

```
Payment Service
  ↓
HTTP/REST
  ↓
Transaction Service
```

Typical technologies:

```
REST
Feign
WebClient
```

Advantages:

- straightforward request/response
- easy to understand

Disadvantages:

- tightly coupled at runtime
- caller waits
- downstream failure affects caller

12. Asynchronous Communication

Example:

```
Payment Service
  ↓
Kafka
  ↓
Transaction Service
```

Payment service can publish:

```
PaymentProcessed
```

and another service consumes it.

Benefits:

- loose runtime coupling
- buffering
- asynchronous processing
- better resilience in some workflows

Costs:

- eventual consistency
- more complicated debugging
- duplicate messages
- ordering concerns
- retries/DLQ

13. Failure Handling in Microservices

Suppose:

```
BPS
↓
Payment Service
↓
Gateway
```

Gateway is down.

A good architecture should think about:

```
timeout
retry
circuit breaker
fallback
idempotency
logging
alerting
reconciliation
```

For payment processing, **blind retries are dangerous**.

Example:

```
Request sent
↓
Gateway processes payment
↓
Response lost
↓
BPS sees timeout
↓
Retry?
↓
Potential duplicate payment
```

So payment systems need careful idempotency and transaction-state handling.

14. Circuit Breaker

Suppose a downstream service keeps failing:

```
BPS
↓
Payment Service
↓
Unavailable Service
```

Without protection:

```
1000 requests
↓
1000 failing calls
```

A circuit breaker can transition:

```
CLOSED
↓
Failures exceed threshold
↓
OPEN
↓
Stop calling downstream
```

Later:

```
OPEN
↓
HALF-OPEN
↓
Test request
↓
Healthy → CLOSED
```

Common Java implementation:

```
Resilience4j
```

Know the concept even if your project didn't use it.

15. API Gateway

Microservice systems frequently use an API Gateway:

```
Frontend
↓
API Gateway
↓
├── Payment Service
├── Customer Service
└── Transaction Service
```

Gateway can handle things such as:

- routing
- authentication integration
- rate limiting
- request filtering
- common policies

Don't assume all systems need a gateway, but understand why one is used.

16. Service Discovery

If services are dynamically deployed:

```
Payment Service
  ↓
"Where is Transaction Service?"
  ↓
Service Discovery
```

Examples historically include:

- Eureka
- Consul
- Kubernetes service discovery/DNS

Modern Kubernetes environments often provide service discovery without requiring a separate application-level registry.

17. Distributed Tracing

This becomes important when debugging.

Monolith:

```
Request
  ↓
Controller
  ↓
Service
  ↓
DB
```

Microservices:

```
Request
  ↓
Gateway
  ↓
Payment Service
  ↓
Transaction Service
  ↓
Notification Service
```

You need correlation/trace IDs:

Trace ID: ABC123

so you can follow the request across services.

Common technologies:

```
OpenTelemetry
Jaeger
Zipkin
```

You don't need to memorize all of them, but understand the purpose.

18. Monitoring

With microservices, monitoring becomes more important because failures can occur anywhere.

You generally track:

```
Latency
Error rate
Traffic
Resource utilization
```

and service-specific metrics.

For payment systems:

```
Payment success rate
Payment failure rate
Gateway timeout rate
Processing latency
```

19. Monolith Does NOT Mean Bad Architecture

A well-designed monolith can have:

```
Clear modules
Loose coupling
Good boundaries
Good testing
Clean architecture
```

The real problem is often a **poorly structured monolith**, not the deployment model itself.

Likewise, microservices can become a **distributed monolith** if:

```
Service A
↓
B
↓
C
↓
```

and every deployment requires all services to change together.

That's one of the best interview points to know.

20. Microservices vs Distributed Monolith

True microservices

A ↔ B
A ↔ C

Independent deployment
Independent scaling
Clear boundaries

Distributed monolith

A → B → C → D → E

Everything is tightly coupled through runtime dependencies.

You get:

- complexity of microservices
- without much of the independence benefit

21. When Would You Choose Monolith?

Choose a monolith when:

- application is small
- team is small
- domain is relatively simple
- rapid initial development is more important
- independent scaling isn't needed
- operational maturity is limited

22. When Would You Choose Microservices?

Consider microservices when:

- business domains are clearly separable
- independent deployment is valuable
- different components need different scaling
- multiple teams need autonomous ownership
- system complexity justifies distributed architecture

Again:

Microservices solve organizational and scaling problems but create distributed-system problems.

That's a strong interview statement.

23. Questions You Should Prepare

Basic

1. What is monolithic architecture?
2. What is microservices architecture?
3. Difference between monolith and microservices?
4. Advantages of microservices?
5. Disadvantages?
6. Why would you choose microservices?

3-year level

7. How do microservices communicate?
8. REST vs Kafka?
9. What happens when one service is unavailable?
10. How do you handle retries?
11. What is circuit breaker?
12. What is API Gateway?
13. What is service discovery?
14. How do services handle authentication?
15. How do you monitor microservices?
16. What is distributed tracing?
17. How do you test microservices?

Database/design

18. Should every microservice have its own DB?
19. Why shouldn't services directly access each other's DB?
20. How do you handle transactions across services?
21. What is Saga?
22. What is eventual consistency?
23. What is the Outbox pattern?

Payment-specific

24. What happens if Payment Service succeeds but Transaction Service fails?
 25. What happens if the payment gateway times out?
 26. How do you prevent duplicate payment?
 27. How do you reconcile inconsistent transaction states?
 28. How do you guarantee idempotency?
-

24. Interview Scenario

Interviewer:

“Suppose payment service calls transaction service. Payment succeeds, but transaction service is down. What do you do?”

Don't say:

“Rollback the payment.”

That's not necessarily possible because the payment may already have been accepted by an external system.

A stronger answer:

```
Payment processing
  ↓
SUCCESS
  ↓
Transaction Service unavailable
  ↓
Persist durable state/event
  ↓
Retry delivery
  ↓
Idempotent consumer
  ↓
Transaction Service processes
  ↓
Consistent final state
```

Depending on architecture, this could involve:

```
Outbox
+
Kafka
+
Idempotent consumer
+
Retry/DLQ
```

The exact solution depends on your system.

25. How You Should Explain Your BPS Architecture

For your interview, don't learn a generic architecture and claim it was yours.

First establish your **actual architecture**:

```
Client
  ↓
API Gateway?
  ↓
BPS
  ↓
Which services?
  ↓
REST/Kafka?
  ↓
Database?
  ↓
```

Then be able to explain:

Why was it designed that way? What did you personally work on? What happens when a downstream system fails? How is consistency maintained? How are transactions traced?

That will be much more valuable than memorizing definitions.

Interview-ready answer

“A monolithic application is typically built and deployed as one unit, with modules such as payment, customer and transaction processing running within the same application. Microservices split the system into independently deployable services aligned to business capabilities. Microservices provide benefits such as independent deployment and scaling, but introduce network latency, distributed failures, observability complexity and data-consistency challenges. In a payment system, the choice is particularly important because cross-service transactions, retries and failures require mechanisms such as idempotency, asynchronous messaging, compensation or reconciliation. I would choose microservices when the business and operational requirements justify that additional complexity rather than using them by default.”

For your preparation, remember this line:

Monolith simplifies distributed complexity; microservices improve independence but introduce distributed complexity.

For your **BPS interview**, the next concepts I would learn immediately after this are **API Gateway, service-to-service communication, Circuit Breaker/Retry, Saga, Eventual Consistency, and Outbox Pattern**.